

Ethernet Communication Overview

Each board in the DAQ must run its own firmware, and communicate with other boards and the server via Ethernet. This requires Ethernet packet structures. As an example (numbers are # of bytes):

Packet Type (1)	Board ID (1)	Data (1 for ID + 4 per data) * number of data points
-----------------	--------------	--

Here is a list of the types of communication that must take place in the DAQ system:

1. “Heartbeats” periodically sent from the server to all boards
 - a. To detect loss of server connection → start abort
 - b. How periodically? Idk we can play around with it
2. “Heartbeats” periodically sent from each board to the server
 - a. To detect if a board goes down / loses connection
3. “Abort” command sent from server to all boards
4. Sensor data from boards (PT, TC, LC, RTD) sent to the server
5. Actuator current data from the actuator board sent to the server

Let’s break that down by board:

Analog Sensor Boards:

Pressure Transducer Board: collects pressure data from 1-10 sensors

Load Cell Board: collects force data from 1-10 LCs during engine firing

RTD Board: collects low temperature data from 1-4 RTDs placed on tanks and lines

TC Board: collects high temperature data using 1-10 TCs on the combustion chamber during firings

All of the analog sensor boards:

- Ignores aborts (all they do is read sensors)
- Continuously sends data from their sensors to the server
 - Ideally if only 5 PTs are connected, we just sent data for those 5, so we need to identify which sensor each datapoint is from
 - Must consider how much data we want to put into each packet, could buffer into few large packets or send many small packets
 - Somehow we need to sync the data for all of the boards and timestamp them, so that the server can “line up” the data between boards
 - Each board has an internal clock that counts how long it has been since startup but I don’t think it’s very accurate

Note that there could be multiple of any of these boards in the system.

Actuator Boards: controls actuators as instructed by the server and send data on the current through each actuator to the server

- So packets from server → board should have a list of valves and what their states should be
- Packets from board → server should be the current through all of the actuators
- In the case of an abort (either due to lost connection or server command) must set actuators in abort state in a specific order
 - If there are multiple actuators boards, they must communicate to ensure the order is satisfied between them. To do this I propose we make one actuator board the “controller” and it does its abort first, then instructs the other boards to do their abort in the correct order.

Board Firmware Overview

The analog boards have relatively simple firmware, since they must simply communicate with their ADCs and send that data out. This is done via SPI. There is one SPI bus connected to the ADCs, and another connected to Ethernet. The ESP32 collects data from the ADC bus and sends it out via the Ethernet bus.

The actuator board controls each actuator using a simple digital output pin and reads the currents using an ADC pin. It also has an SPI bus connected to Ethernet, but that is its only SPI bus.

Proposed Ethernet Packet Structure

Header

All packet types start with the header:

Packet Type (1)	Version (1)	Timestamp (4)
-----------------	-------------	---------------

Timestamp should be precise down to 0.1mS. Exact format is not yet determined.

Packet types:

ID	Name
1	Board Heartbeat
2	Server Heartbeat
3	Sensor Data
4	Actuator Command
5	Sensor Config
6	Actuator Config
7	Abort
8	Abort Done
9	Clear Abort

Board Heartbeat

Board Type and ID (1)	Engine State (1)	Board State (1)
-----------------------	------------------	-----------------

Engine state refers to the state of the engine (ie LOX fill) and is sent in the heartbeat so that the server can ensure that boards know the correct state. The board state refers to the board state machine.

Board State

ID	Name
1	Setup
2	Active
3	Abort
4	Abort done

Server Heartbeat

Engine State (1)

This is how the server communicates the state to each board. When this is received the board should keep track of the current state.

Sensor Data

# Chunks (1)	# Sensors connected (1)	N chunks ($N * (5S + 4)$)
--------------	-------------------------	-----------------------------

Where N is the number of chunks, and S is the number of sensors connect.

Each **Chunk** is of the format:

Timestamp (4)	S Sensor Datapoints (5S)
---------------	--------------------------

Each **Sensor Datapoint** is of the format:

Sensor ID (1)	Sensor data (4)
---------------	-----------------

Sensor ID maps to the connectors on the board, 1-indexed, from left-to-right, top-to-bottom.

Actuator Command

# Commands (1)	N commands (2N)
----------------	-----------------

Where N is the number of commands.

Each **Command** is of the format:

Actuator ID (1)	Actuator State (1)
-----------------	--------------------

Actuator ID maps to the connectors on the board, 1-indexed, from left-to-right, top-to-bottom.

Sensor Config

# Sensors (1)	N sensor IDs (N)	Necessary for abort? (1)	Controller IP address (4)
---------------	------------------	--------------------------	---------------------------

Where N is # sensors. The first two fields are to tell the board what sensors the server wants data from. Necessary for abort determines if the board needs to provide data to the controlled in the case of an abort.

Note that the Controller IP address will be left empty if necessary for abort is false.

Actuator Config

Is abort controller? (1)	N Abort Actuator Locations (6N)	X Abort PT Locations (6X)
--------------------------	---------------------------------	---------------------------

Where N is the number of actuators involved in abort, hardcoded, and where X is the number of PTs involved in abort, also hardcoded.

Each **Abort Actuator Location** is of the format

IP address (4)	Actuator ID (1)	What is it for? (1) (purpose)
----------------	-----------------	-------------------------------

“What is it for?” is there so that the controller knows which actuator is which, so it can do them in the correct order. The point of this field is to track where each actuator is connected, in case it is all on a board other than the controller. Actuator ID is the same as the Actuator ID field in the data packet, it is physically which port the sensor is connected to. It is necessary because the sense boards do not know which sensors are needed for abort and it will send them all, then the controller will just process the data from the sensors it cares about.

Each **Abort PT Location** is of the format

IP address (4)	Sensor ID (1)	What is it for? (1)
----------------	---------------	---------------------

The format of this field is essentially the same as the Abort Actuator Location field above.

RTD Board

Table 10-2. RTD Circuit Design Parameters

I_{IDAC} (μA)	NFR (bits)	P_{RTD} (mW)	$V_{RTD}^{(1)}$ (V)	Gain ⁽²⁾ (V/V)	$V_{REFMIN}^{(3)}$ (V)	$V_{REF}^{(4)}$ (V)	$R_{REF}^{(5)}$ (k Ω)	$V_{INNLIM}^{(6)}$ (V)	$V_{INPLIM}^{(7)}$ (V)	$R_{BIAS}^{(8)}$ (k Ω)	$V_{RTDN}^{(9)}$ (V)	$V_{RTDP}^{(10)}$ (V)	$V_{IDAC1}^{(11)}$ (V)
50	16.8	0.001	0.02	32	0.64	0.90	18	0.6	4.1	7.10	0.7	0.7	1.9
100	17.8	0.004	0.04	32	1.28	1.41	14.1	0.9	3.8	5.10	1.0	1.1	2.8
250	18.8	0.025	0.10	16	1.60	1.76	7.04	1.1	3.7	2.30	1.2	1.3	3.3
500	19.1	0.100	0.20	8	1.60	1.76	3.52	1.0	3.8	1.10	1.1	1.3	3.4
750	18.9	0.225	0.30	4	1.20	1.32	1.76	0.8	4.0	0.57	0.9	1.2	2.8
1000	19.3	0.400	0.40	4	1.60	1.76	1.76	0.9	3.9	0.50	1.0	1.4	3.5
1500	19.1	0.900	0.60	2	1.20	1.32	0.88	0.6	4.2	0.23	0.7	1.3	3.0
2000	18.3	1.600	0.80	1	0.80	0.90	0.45	0.3	4.5	0.10	0.4	1.2	2.4

- (1) V_{RTD} is the RTD input voltage.
- (2) Gain is the ADC gain
- (3) V_{REFMIN} is the minimum reference voltage required by the design.
- (4) V_{REF} is the design target reference voltage allowing for 10 % over-range or the minimum 0.9 V reference voltage requirement.
- (5) R_{REF} is the resistor that senses the I_{IDAC} current to generate V_{REF} .
- (6) V_{INNLIM} is the absolute minimum input voltage required by the ADC.
- (7) V_{INPLIM} is the absolute maximum input voltage required by the ADC.
- (8) R_{BIAS} establishes the level-shift voltage.
- (9) V_{RTDN} is the design target negative input voltage.
- (10) V_{RTDP} is the design target positive input voltage.
- (11) V_{IDAC1} is the design target I_{IDAC1} loop voltage.

MAG1[3:0] bits 3:0 of $I_{IDACMAG}$
(register address = 0Eh)

0000: off
 0001: 50 μA
 0010: 100 μA
 0011: 250 μA
 0100: 500 μA
 0101: 750 μA
 0110: 1000 μA
 0111: 1500 μA
 1000: 2000 μA
 1001: 2500 μA
 1010: 3000 μA